

Session 4

Structs & State Machines



Keep related values together, and give your program a clear set of moods.

Part	Topic	Time
01	struct: one card for related values	8 min
02	State machines: one state at a time	8 min
03	Practice: a traffic light	9 min
04	AI integration: review your state machine	5 min

Builds on functions and switch from Session 2, and the sensor readings from Lesson 3.

Six words you will hear today.

Word	Plain meaning
struct	A new type you design yourself, that groups several values under one name.
Member	One value inside a struct, reached with a dot: d.temperature.
State	What the program is doing right now — like "red" for a traffic light.
Transition	Moving from one state to the next.
State machine (FSM)	A program that is always in exactly one state and moves between states by clear rules.
enum	A list of named values, like RED, YELLOW, GREEN, so code says names instead of numbers.

A struct is a student ID card for your data.

Name, class and number belong together, so they are printed on one card — not on three loose slips of paper.

Three readings that belong together.

ONE LINE OF CODE

```
float temperature = incipe.getTemperature();  
float humidity = incipe.getHumidity();  
float light = incipe.getLightIntensity();
```

Call	Value	Not detected	Read it every
<code>incipe.getTemperature()</code>	A float in degrees Celsius (°C)	-1	2000 ms
<code>incipe.getHumidity()</code>	A float, % relative humidity, 0–100	-1	2000 ms
<code>incipe.getLightIntensity()</code>	A float from 0 (dark) to 1023 (bright)	-1	20–500 ms

Full details: [Wiki › Temperature & humidity sensor](#), [Wiki › Light sensor](#).

Design the card, then fill it in.

```
1 struct SensorData {  
2     float temperature;    // °C  
3     float humidity;      // %  
4     float light;         // 0-1023  
5 };  
6  
7 SensorData readAll() {  
8     SensorData d;  
9     d.temperature = incipe.getTemperature();  
10    d.humidity = incipe.getHumidity();  
11    d.light = incipe.getLightIntensity();  
12    return d;  
13 }
```

LINE 1 · STEP 1 OF 9

Design a new type called SensorData — the blank card.

THE CARD D

Not made yet.

Design the card, then fill it in.

```
1 struct SensorData {  
2     float temperature;    // °C  
3     float humidity;      // %  
4     float light;         // 0-1023  
5 };  
6  
7 SensorData readAll() {  
8     SensorData d;  
9     d.temperature = incipe.getTemperature();  
10    d.humidity = incipe.getHumidity();  
11    d.light = incipe.getLightIntensity();  
12    return d;  
13 }
```

LINE 2 · STEP 2 OF 9

It has three members. Each is a float, with a comment saying its unit.

THE CARD D

Not made yet.

Design the card, then fill it in.

```
1 struct SensorData {  
2     float temperature;    // °C  
3     float humidity;       // %  
4     float light;         // 0-1023  
5 };  
6  
7 SensorData readAll() {  
8     SensorData d;  
9     d.temperature = incipe.getTemperature();  
10    d.humidity = incipe.getHumidity();  
11    d.light = incipe.getLightIntensity();  
12    return d;  
13 }
```

LINE 5 · STEP 3 OF 9

The ; after } ends the design. Nothing is stored yet.

THE CARD D

Not made yet.

Design the card, then fill it in.

```
1 struct SensorData {
2     float temperature;    // °C
3     float humidity;       // %
4     float light;         // 0-1023
5 };
6
7 SensorData readAll() {
8     SensorData d;
9     d.temperature = incipe.getTemperature();
10    d.humidity = incipe.getHumidity();
11    d.light = incipe.getLightIntensity();
12    return d;
13 }
```

LINE 7 · STEP 4 OF 9

A function that hands back a whole SensorData.

THE CARD D

Not made yet.

Design the card, then fill it in.

```
1 struct SensorData {
2     float temperature;    // °C
3     float humidity;       // %
4     float light;          // 0-1023
5 };
6
7 SensorData readAll() {
8     SensorData d;
9     d.temperature = incipe.getTemperature();
10    d.humidity = incipe.getHumidity();
11    d.light = incipe.getLightIntensity();
12    return d;
13 }
```

LINE 8 · STEP 5 OF 9

Make one empty card, d.

THE CARD D

```
d.temperature = ?
d.humidity = ?
d.light = ?
```

Design the card, then fill it in.

```
1 struct SensorData {
2     float temperature;    // °C
3     float humidity;       // %
4     float light;         // 0-1023
5 };
6
7 SensorData readAll() {
8     SensorData d;
9     d.temperature = incipe.getTemperature();
10    d.humidity = incipe.getHumidity();
11    d.light = incipe.getLightIntensity();
12    return d;
13 }
```

LINE 9 · STEP 6 OF 9

The dot reaches inside: fill in the temperature. Say it is 23.4.

THE CARD D

```
d.temperature = 23.4
d.humidity = ?
d.light = ?
```

Design the card, then fill it in.

```
1 struct SensorData {
2     float temperature;    // °C
3     float humidity;       // %
4     float light;         // 0-1023
5 };
6
7 SensorData readAll() {
8     SensorData d;
9     d.temperature = incipe.getTemperature();
10    d.humidity = incipe.getHumidity();
11    d.light = incipe.getLightIntensity();
12    return d;
13 }
```

LINE 10 · STEP 7 OF 9

Fill in the humidity: 51.

THE CARD D

d.temperature = 23.4

d.humidity = 51.0

d.light = ?

Design the card, then fill it in.

```
1 struct SensorData {
2     float temperature;    // °C
3     float humidity;      // %
4     float light;         // 0-1023
5 };
6
7 SensorData readAll() {
8     SensorData d;
9     d.temperature = incipe.getTemperature();
10    d.humidity = incipe.getHumidity();
11    d.light = incipe.getLightIntensity();
12    return d;
13 }
```

LINE 11 · STEP 8 OF 9

Fill in the light: 700.

THE CARD D

```
d.temperature = 23.4
d.humidity = 51.0
d.light = 700.0
```

Design the card, then fill it in.

```
1 struct SensorData {
2     float temperature;    // °C
3     float humidity;       // %
4     float light;         // 0-1023
5 };
6
7 SensorData readAll() {
8     SensorData d;
9     d.temperature = incipe.getTemperature();
10    d.humidity = incipe.getHumidity();
11    d.light = incipe.getLightIntensity();
12    return d;
13 }
```

LINE 12 · STEP 9 OF 9

Hand back the whole card in one go.

THE CARD D

```
d.temperature = 23.4
d.humidity = 51.0
d.light = 700.0
```

One call, one card, every value.

```
1  SensorData now = readAll();  
2  Serial.print(now.temperature);  
3  Serial.print(" ");  
4  Serial.println(now.light);
```

SERIAL MONITOR

23.40 700.00

With the values from the walkthrough.

STILL CHECK FOR -1

Each member can be -1 if its sensor is not detected. Check `now.temperature >= 0` before you use it.

WHY BOTHER?

One function returns all three values. An array of `SensorData` keeps a whole log in order.

SIZE

12 bytes on the INCIPE Board: three floats of 4 bytes each.

A state machine is a traffic light.

At any moment it shows exactly one colour — never two, never none. After a set time it moves to the next colour, always in the same order.

A fixed list of states, and clear rules for each transition. FSM = finite state machine.

Give each state a name.

```
1 enum Light { RED, YELLOW, GREEN };  
2 Light state = RED;
```

Line	What it does
1	A new type Light that can only be RED, YELLOW or GREEN.
2	The light starts in the RED state.

The code now reads RED instead of a mystery number.

Write the machine as a table before you code it.

State	Shows	Lasts	Next state
RED	Red	3000 ms	YELLOW
YELLOW	Yellow	1000 ms	GREEN
GREEN	Green	3000 ms	RED

The curriculum order: Red → Yellow → Green → Red. Each row of the table becomes one case.

Real traffic lights differ from country to country. Change the table, and the code follows.

The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 1 · STEP 1 OF 8

Three named states.

VARIABLES

state = ?

SERIAL MONITOR



The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 2 · STEP 2 OF 8

Start in RED.

VARIABLES

state = RED

SERIAL MONITOR

The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 5 · STEP 3 OF 8

First pass of loop(): which state are we in? RED.

VARIABLES

state = RED

SERIAL MONITOR



The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 6 · STEP 4 OF 8

Show red, wait 3 s, then move to YELLOW. break leaves the switch.

VARIABLES

state = YELLOW

SERIAL MONITOR

RED

The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 5 · STEP 5 OF 8

Next pass: state is YELLOW.

VARIABLES

state = YELLOW

SERIAL MONITOR

RED

The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 7 · STEP 6 OF 8

Show yellow for 1 s, then move to GREEN.

VARIABLES

state = GREEN

SERIAL MONITOR

RED
YELLOW

The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 5 · STEP 7 OF 8

Next pass: GREEN.

VARIABLES

state = GREEN

SERIAL MONITOR

RED
YELLOW

The traffic light, one pass at a time.

```
1  enum Light { RED, YELLOW, GREEN };
2  Light state = RED;
3
4  void loop() {
5      switch (state) {
6          case RED:    Serial.println("RED");    delay(3000); state = YELLOW; break;
7          case YELLOW: Serial.println("YELLOW"); delay(1000); state = GREEN;  break;
8          case GREEN:  Serial.println("GREEN");  delay(3000); state = RED;    break;
9      }
10 }
```

LINE 8 · STEP 8 OF 8

Show green for 3 s, then back to RED — the cycle repeats.

VARIABLES

state = RED

SERIAL MONITOR

RED
YELLOW
GREEN

Run it, extend it, count it.

Step	Do this
1	Add <code>setup()</code> with <code>Serial.begin(9600);</code> , Upload, and time one full cycle.
2	Add a fourth state, <code>RED_YELLOW</code> , between <code>RED</code> and <code>GREEN</code> (shown as "RED + YELLOW" for 1 s), as many countries use. Update the table first, then the code.
3	Keep a count of how many full cycles have run, and print it every time the light turns red.

On the LED strip: that version will be added once the Wiki page lists what `incipe.setLEDcolour(...)` and `incipe.showLED()` take. Each `Serial.println` is where the LED call will go.

Ask the AI to review your state machine.

PASTE YOUR CODE INTO THE WORKSPACE AI CHAT WITH

Review this traffic-light state machine. Check that every state has exactly one next state and that no state can be skipped or reached by mistake. Suggest up to three improvements and explain each in one sentence for a beginner. Do not rewrite the whole program.

The curriculum task: have AI review your FSM logic and suggest improvements.

Check what comes back like a reviewer.

Ask yourself	Why
Does every suggested change still match my state table?	The table is the plan. Change the table first, then the code.
Did it add a default: case?	A good idea: if state ever holds something unexpected, go back to a safe state like RED.
Did it suggest millis() instead of delay()?	Useful later: with delay() the board cannot read a button during a light. M3 Session 4 covers it.
Did it rename my incipe.* calls or invent new ones?	Never accept that — compare with the Wiki.

How many bytes is one SensorData?

READ THE CODE

```
struct SensorData {  
    float temperature;  
    float humidity;  
    float light;  
};
```

ANSWER

12

WHY

Three floats of 4 bytes each on the INCIPE Board.

What prints?

READ THE CODE

```
SensorData x;  
x.temperature = 21.5;  
Serial.println(x.temperature);
```

ANSWER

21.50

WHY

The dot reaches the temperature member; Arduino prints a float with two decimal places.

The light starts in RED. What are the first four lines?

READ THE CODE

```
case RED:    ... state = YELLOW;  
case YELLOW: ... state = GREEN;  
case GREEN:  ... state = RED;
```

ANSWER

RED
YELLOW
GREEN
RED

WHY

Each pass shows the current state, then moves on; after GREEN it goes back to RED.

Which state comes straight after YELLOW?

READ THE CODE

```
case YELLOW:  
  Serial.println("YELLOW");  
  delay(1000);  
  state = GREEN;  
  break;
```

ANSWER

GREEN

WHY

The YELLOW case sets state = GREEN.

How long is one full Red → Yellow → Green cycle?

READ THE CODE

```
case RED:    ... delay(3000); ...  
case YELLOW: ... delay(1000); ...  
case GREEN:  ... delay(3000); ...
```

ANSWER

7 s

WHY

$3000 + 1000 + 3000 = 7000 \text{ ms.}$

Three things to take away.

1

A struct groups related values under one name; reach each member with a dot, and return the whole card from a function.

2

A state machine is always in exactly one state; plan the states and transitions as a table, then write one case per row.

3

An enum gives states readable names — and the AI's review should improve your table, not replace it.

Next — M3 Session 1: Analog sensors & ADC. How the board turns a voltage into a number, and a live Serial Monitor dashboard.